

My Game Design Document

SLHE Game Programming - a living document you'll update all year

VIDEO GAME CREATION

Student Name:

Game Title:

Engine / Platform:

Started (Lesson #/Date):

How to Use This Document

This is not a one-time assignment - it's a notebook for your game that grows alongside it. You already started this in Lesson 2; this Lesson 8 update carries your earlier answers' shape forward with this week's goals added in, so you can keep going in one place.

- There are no wrong answers this early. If you don't know yet, write your best guess - you can change it later.
- Sections marked "YOUR NOTES" are blank on purpose. Everything else is either a fill-in-the-blank or something already planned for the whole class that's included here for reference.
- Each lesson from here on, look for a "Lesson N Goals" checklist inside Levels / Structure - that's this week's specific Level 1/2/3 targets, carried over from your Handout so you don't have to copy them by hand. Check off where each one stands for you.
- Keep this file (or a printed copy) somewhere you'll find it again - you'll be asked to revisit it at several points across both semesters, especially before the Semester 1 capstone and the CoOp Expo.

The Big Picture

Genre

Examples: platformer, top-down adventure, puzzle, shooter, racing, simulation...

One-Sentence Pitch

Example: "A robot has to reach the top of a crumbling tower before it collapses."

Games it's inspired by

What games (or movies, books, etc.) gave you this idea?

Core Gameplay

By Lesson 8, your game has a real lose condition - a lives system that runs out and ends in defeat, not just a win to chase. Everything from here through Lesson 11 keeps building on this same loop: scoring, a real UI, restart flow, and multiple waves.

Core Mechanic

What's the ONE thing the player does over and over? Everything else in your game supports this one action.

Controls

Pre-filled with your controls from Lessons 2 and 3 - update the "Still accurate?" column (or add rows) as your game grows.

Input	Action	Still accurate?
Right Arrow	Move right (from your Lesson 2 Key Press Reactor)	
Left Arrow	Move left	
Up Arrow	Move up	
Down Arrow	Move down	
Spacebar (or Up Arrow)	Shoot a projectile upward (introduced Lesson 3)	

Win Condition

How does the player succeed?

Lose Condition

How does the player fail? (Not every game needs one!)

World & Characters

Player Character

Who / what is the player controlling?

Setting / World

Where does this take place?

Other Characters, Enemies, or NPCs

Levels / Structure

Reminder - how "Level 1 / 2 / 3" works in this class:

Level	Think of it as...	What it means
Level 1 - Minimal	MVP	The smallest working version - enough to understand the idea and move on. A complete, legitimate finish.
Level 2 - Complete	Internal Alpha/Beta	All expected features present and working end-to-end. Art/polish/balance may still be missing.
Level 3 - Refined	Ready to Publish	Feature-complete, tested, and polished enough to comfortably share outside of class.

Lesson 8 Goals

This week's specific Level 1/2/3 goals - the same goals for every engine (see your Handout for exactly how to build each one in the engine you picked). Check off where each one stands for you right now; it's fine to come back and update this later.

This Lesson's Goal	Plan to Implement	Implemented	Not Planned
Level 1: every enemy that reaches the bottom unshot costs the player one life (tracked starting at 3); "Game Over" appears the instant lives reach zero.	☐	☐	☐
Level 2: shooting an enemy never costs a life, and letting one escape never counts as a kill - the two counters stay	☐	☐	☐

This Lesson's Goal	Plan to Implement	Implemented	Not Planned
completely independent.			
Level 3 - Visual: add a live, continuously-updating lives readout using your engine's simplest text tool.	☐	☐	☐
Level 3 - Sound: play a distinct sound the moment a life is lost.	☐	☐	☐
Level 3 - Structure: keep the lives/escape logic and the kills/hit-detection logic in clearly separate places, with neither one referencing the other's counter - the same separation principle used since Lesson 6.	☐	☐	☐

Level/Area	What happens there	Status

Visual Art Plan

Placeholder art is completely fine while you're prototyping - the lesson Assets folders always include ready-to-use placeholder sprites so you never have to stop and hunt for art mid-lesson. Use this section to describe what you eventually want your game to look like.

Art Style

Examples: pixel art, flat cartoon, simple geometric shapes, hand-drawn...

Color Palette / Mood

Placeholder assets currently in use (check any that apply)

- ☐ player_circle_blue.png / player_circle_green.png / player_circle_orange.png
- ☐ player_arrow_white.png
- ☐ player_walk_spritesheet.png
- ☐ bullet_yellow.png (introduced Lesson 3)
- ☐ enemy_invader_red.png (introduced Lesson 4)
- ☐ My own art (describe below)

Notes on your own art assets

Audio Plan

sfx_blip.wav, sfx_shoot.wav, and sfx_hit.wav are the placeholder sound effects provided in early lessons. Three placeholder sound effects are provided in this lesson's Assets folder for Level 3's "lost a life" sound idea: sfx_lifelost.wav (play it each time a life is lost), sfx_gameover.wav (play it when lives reaches zero), and sfx_start.wav (the game-start stinger from Lesson 6, repeated for convenience).

Music Style / Mood

Sound Effects Needed

List the moments in your game that should make a sound.

Scope & Milestones

This is the class's actual 30-lesson schedule (see the Course Syllabus for full detail) with a column for your own status. Use it to check whether your game is on pace with the rest of the class, not as a strict deadline - talk to your instructor if you're falling behind or racing ahead.

Lessons	Class Focus	Class Milestone	My Status
1	Orientation: internet safety, game dev overview, platform selection	Engine installed & first project opened	
2-6	Programming foundations: variables, events, loops, conditionals, input	First moving, controllable character	
7-11	Core mechanics: collision, scoring, win/lose states, simple UI	Small playable prototype	
12-15	Game feel & polish: audio, basic animation, level structure	Semester 1 capstone: complete Mini-Game	
16-20	Intermediate systems: state machines, enemy/AI behavior, physics	Prototype with opponents/obstacles	
21-25	Expanding scope: menus, save data, multiple levels, juice/polish	Feature-complete draft of capstone game	
26-28	Playtesting, bug-fixing, presentation prep	Playtested, stable build	
29-30	Finalize, package, and export shareable copies	Shareable build ready for CoOp Expo	

Semester 1 Capstone (Lessons 12-15)

By the end of Lesson 15, your game should be a complete Mini-Game: it has a clear core mechanic, art and audio (even simple/placeholder), and reaches a real win or lose state. This is the same project you started in Lesson 2 - it's been growing the whole time.

My plan for reaching the Semester 1 capstone

Semester 2 / CoOp Expo Capstone (Lessons 26-30)

The final weeks of Semester 2 shift from new material to finishing, playtesting, and packaging your game so it can be demonstrated at the co-op's Expo and shared with friends and classmates. You'll export a runnable build (or shareable link, for browser-based engines) and prepare a short demo/pitch, with structured playtesting feedback from classmates before the Expo.

My plan for the CoOp Expo

The Five Disciplines of Game Development

Every game (including yours) touches all five of these, even on a small student project. Programming is our main focus in class, but the other four matter too - use the space under each to jot ideas as they come to you.

Design

Deciding what the game is and how it plays - the rules, the goals, the feel.

YOUR NOTES

Programming

Making the rules and mechanics actually work in code (our main focus this year).

YOUR NOTES

Visual Art

Characters, environments, and UI - what the player sees.

YOUR NOTES

Audio

Music and sound effects - what the player hears.

YOUR NOTES

Production

Planning, scoping, and finishing a project - relevant all year, critical before the Expo.

YOUR NOTES

Design Journal / Change Log

A quick running log of significant changes to your game and why you made them - useful for remembering your own reasoning later, and required reading for yourself before the Semester 1 and Expo capstones.

Date	Lesson #	What changed and why

Date	Lesson #	What changed and why